

Betsy -- Pipeline Architecture

Linear · Sequential · Single-threaded · Audit-first One stage feeds the next. All conditions detected before any action is taken. LLM (local Ollama) handles judgment calls. Rule-based fallback if model is unavailable.

Overview

```
Trigger ??? INGEST ??? DETECT ??? EVALUATE ??? DECIDE ??? [APPROVAL] ??? ACT ??? AUDIT
              (1)          (2)          (3) ?LLM      (4) ?LLM              (5)          (6) ?LLM
```

Each stage transforms a shared PipelineContext object. Stages cannot skip or reorder. A PipelineHalt exception short-circuits to Stage 6 (Audit always runs).

Entry Point -- pipeline/run.py

```
run_pipeline(
    triggered_by = "manual" | "scheduler" | "test"
    api_base_url = "http://localhost:8000"
    ollama_base_url = env:OLLAMA_BASE_URL -> default: http://localhost:11434
    ollama_model    = env:OLLAMA_MODEL    -> default: mistral
    approval_mode   = "console" | "auto_approve" | "auto_reject" | "webhook"
)
-> PipelineContext
```

Creates: PipelineContext · BetsyClient · LLMClient · ApprovalGate

Runs stages in order. Intercepts PipelineHalt -> jumps to Audit.

Shared State -- PipelineContext (pipeline/context.py)

```
PipelineContext
??? run metadata:      run_id · started_at · triggered_by · scenario
?
??? Stage 1 output:    inventory[] · all_pos[] · open_pos[] · invoices[] · suppliers[]
?
??? Stage 2 output:    detected_conditions[]
?                      DetectedCondition:
?                        type          -> "stockout" | "price_spike" | "duplicate_invoice"
?                        severity      -> "critical" | "warning" | "info"
?                        sku_id        -> str | None
?                        data          -> {raw numbers: days_remaining, pct_above_avg, ...}
?
??? Stage 3 output:    evaluated_items[]
?                      EvaluatedItem:
?                        condition      -> DetectedCondition
?                        ranked_options[] -> [{supplier_id, score, unit_price, lead_days}]
?                        best_option    -> dict | None
?                        llm_reasoning  -> str (LLM explanation of recommendation)
?                        confidence     -> float 0.0-1.0
?
??? Stage 4 output:    decisions[]
?                      Decision:
```

```

?          action          -> "generate_po" | "flag_duplicate" |
?          "flag_for_approval" | "escalate" | "no_action"
?          auto_approved   -> bool
?          requires_human   -> bool
?          llm_reasoning    -> str (LLM explanation of decision)
?          confidence       -> float
?          metadata         -> dict
?
??? Stage 5 output:  actions_taken[]
?                    ActionResult:
?                    decision    -> Decision
?                    api_endpoint -> str
?                    api_payload -> dict
?                    response    -> dict | None
?                    success     -> bool
?                    error       -> str | None
?
??? Stage 6 output:  audit_written · final_status · halt_reason

```

Stage 1 -- Ingest (pipeline/stages/ingest.py)

Role: Fetch all data the pipeline needs in one pass. No decisions.

Functions

?????????

run(ctx, client) -> PipelineContext

client.get_inventory()	??? ctx.inventory	(12 SKUs)
client.get_purchase_orders()	??? ctx.all_pos	(full PO history)
	ctx.open_pos	(pending only, pre-filtered)
client.get_invoices()	??? ctx.invoices	(14 invoices)
client.get_suppliers()	??? ctx.suppliers	(6 suppliers)
client.get_active_scenario()	??? ctx.scenario	

API calls: GET /api/inventory
 GET /api/purchase-orders
 GET /api/invoices
 GET /api/suppliers

Halt: any request fails -> PipelineHalt("API unavailable")
 (Audit stage still runs -- partial context is logged)

LLM: none

Stage 2 -- Detect (pipeline/stages/detect.py)

Role: Find ALL anomalies in ingested data.

No short-circuit. No LLM. Purely rule-based.

ALL conditions are collected before any evaluation begins.

Functions

?????????

run(ctx, client) -> PipelineContext

_detect_duplicate_invoices(invoices)

```
Algorithm: group by (supplier_id, amount)
          flag pairs where date_diff ? 60 days
Produces:  DetectedCondition(type="duplicate_invoice", severity="warning")
data keys: invoice_1_id, invoice_2_id, amount, days_apart, risk_level
```

```
_detect_price_spikes(inventory, all_pos, suppliers, threshold=0.18)
```

```
Algorithm: baseline    = inventory[i].unit_cost_avg
          best_quote   = min quote from available suppliers for this SKU
          spike if     best_quote > baseline × 1.18
Produces:  DetectedCondition(type="price_spike", severity="warning")
data keys: sku_id, best_quote, baseline, pct_above, threshold
```

```
_detect_stockouts(inventory)
```

```
Algorithm: flag if current_stock < reorder_point
          days_remaining = current_stock / daily_usage_avg
          severity = "critical" if days_remaining < 2.0 else "warning"
Produces:  DetectedCondition(type="stockout", severity="critical|warning")
data keys: current_stock, reorder_point, daily_usage_avg, days_remaining, max_stock
```

```
API calls: none
LLM:       none
```

```
Output:    ctx.detected_conditions[] (may be empty -- no conditions found is valid)
```

Stage 3 -- Evaluate (pipeline/stages/evaluate.py) ? LLM

Role: For each detected condition, score available options.
LLM explains the best choice (supplier, risk level).
Always has a rule-based fallback if Ollama is unreachable.

Functions

?????????

```
run(ctx, client, llm) -> PipelineContext
```

```
_evaluate_stockout(condition, suppliers, llm)
```

Step 1 (rule):

```
filter suppliers where availability=True AND sku_id in catalog
score = reliability_score / lead_days    (formula from tests/scenario_runner.py)
sort descending -> ranked_options
```

Step 2 (LLM call):

```
SYSTEM: "You are a procurement agent. Return JSON only:
        {recommended_supplier_id, confidence, reasoning}"
```

```
USER:   "SKU: {sku_id} ({name})
        Stock: {current_stock} | Reorder: {reorder_point} | Days left: {days_remaining}
        Ranked suppliers:
        {json(ranked_options)}
        Which supplier should we order from and why?
        Consider lead time vs reliability vs price."
```

```
-> llm_reasoning (str), confidence (float)
```

```
Fallback: top rule-based score wins, llm_reasoning="fallback: ollama_unavailable"
```

```
Output: EvaluatedItem(ranked_options, best_option=top_scorer, llm_reasoning, confidence)
```

```

_evaluate_price_spike(condition, suppliers, llm)
    Rule:          rank by price ascending (informational context for human reviewer)
    LLM call:      none (price spikes always go to human -- confidence hardcoded to 0.0)
    Output:       EvaluatedItem(ranked_options, confidence=0.0)

_evaluate_duplicate(condition, llm)
    LLM call:
        SYSTEM: "You are a financial auditor. Return JSON only:
                  {risk_level: HIGH|MEDIUM|LOW, confidence, fraud_likelihood, reasoning}"
        USER:   "Duplicate invoice pairs:
                  {json(pairs)}
                  Are these billing errors or potential fraud?"
    confidence: 1.0 if risk_level=HIGH, 0.7 if MEDIUM
    Output:     EvaluatedItem(best_option={action:"flag"}, llm_reasoning, confidence)

API calls: GET /api/suppliers/{id}/quote (to get live quotes for stockout evaluation)
LLM:      v stockout evaluation + duplicate risk assessment

Output:   ctx.evaluated_items[] (one per detected condition)

```

Stage 4 -- Decide (pipeline/stages/decide.py) ?

LLM

Role: Convert evaluated items into concrete decisions.
 LLM provides action recommendation + confidence + reasoning.
 Code enforces financial safeguards -- LLM cannot override these.

Financial safeguards (hard-wired constants):
 MAX_AUTO_APPROVE_USD = \$5,000 (single PO ceiling)
 MAX_DAILY_SPEND_USD = \$15,000 (aggregate per run)

Functions

?????????

run(ctx, client, llm) -> PipelineContext

```

_decide_for_item(item, accumulated_spend, llm)
    LLM call:
        SYSTEM: "You are an autonomous procurement agent. Return JSON only:
                  {action, confidence, reasoning, requires_human}
                  Allowed actions: generate_po | flag_for_approval |
                                  flag_duplicate | escalate | no_action
                  You MUST follow: price spikes >18% need human approval.
                                  Duplicates always require human review.
                                  No auto-approve above $5,000."
        USER:   "Condition: {json(condition)}
                  Evaluated options: {json(evaluated_item)}
                  Accumulated spend this run: ${accumulated_spend}"
    -> action, confidence, reasoning, requires_human (from LLM)

```

Code overrides (always applied after LLM response):

type=duplicate_invoice	-> requires_human = True (always)
type=price_spike	-> requires_human = True (always)
best_option is None	-> action = "escalate"
po_total > \$5,000	-> requires_human = True

```

    accumulated_spend > $15,000 -> requires_human = True

Fallback (Ollama unreachable):
    duplicate_invoice -> flag_duplicate,          requires_human=True
    price_spike       -> flag_for_approval,      requires_human=True
    stockout          -> generate_po,            auto_approved if spend OK
    no_supplier        -> escalate,              requires_human=True

_compute_order_qty(condition)
    qty = max_stock - current_stock
    fallback: 2 × reorder_point if max_stock not set

API calls:  none
LLM:        v action + confidence + reasoning per decision

Output:     ctx.decisions[]

```

Approval Gate -- shared/approvals.py

Runs between Stage 4 and Stage 5.
 Blocks for decisions where requires_human=True.
 Presents LLM reasoning from Stages 3 & 4 to the human reviewer.

Class: ApprovalGate(mode="console" | "auto_approve" | "auto_reject" | "webhook")

Methods

???????

request_approval(decision) -> ApprovalResult(approved, reviewer, notes, timestamp)

_console_prompt(decision)

Prints:

```

    ?????????????????????????????????????????????????????????????
    ? APPROVAL REQUIRED                                           ?
    ? Action:  flag_for_approval                                  ?
    ? SKU:     SKU-003 (Steel Rods 10mm)                         ?
    ? Reason:  Price spike -- $13.60 is 19% above $11.40        ?
    ? LLM:     "Current quotes exceed threshold. Holding        ?
    ?           purchase recommended."                           ?
    ? Options: QuickShip $15.50 | FastParts $20.00              ?
    ? Approve? [y/N] (60s timeout -> N)                          ?
    ?????????????????????????????????????????????????????????????

```

Reads y/n from stdin. Timeout = 60s -> auto-reject (safe default).

_auto_approve(decision)

Used in test runs. Always approves. Reviewer = "auto_test".

_webhook_request(decision)

POST to configured URL. Poll for response. Raises TimeoutError if no reply.

Effect:

```

approved -> decision.auto_approved = True + metadata.approved_by, approval_notes
rejected -> decision.auto_approved = False + metadata.rejected_by, rejection_notes

```

Stage 5 -- Act (pipeline/stages/act.py)

Role: Execute only auto-approved decisions via API.

Decisions still requiring human sign-off are recorded but NOT executed.

Every API write is wrapped in try/except -- failure logged, pipeline continues.

Functions

?????????

run(ctx, client) -> PipelineContext

```
_execute_generate_po(decision, client)
    POST /api/purchase-orders
        {supplier_id, sku_id, quantity, unit_price,
          reason=decision.reason, requested_by="betsy-pipeline"}
    PATCH /api/purchase-orders/{po_id}/status -> "approved"
    -> ActionResult(success=True, api_response={po_id, ...})
    On error: ActionResult(success=False, error=str(e)) -- pipeline does NOT halt
```

```
_execute_flag_duplicate(decision, client)
    No API write (the flag lives in the audit log)
    -> ActionResult(endpoint="audit_only", success=True)
```

```
_skip_requires_human(decision)
    Creates explicit record: action was intentionally deferred
    -> ActionResult(endpoint="pending_human_review", success=True)
```

API calls: POST /api/purchase-orders
 PATCH /api/purchase-orders/{id}/status

LLM: none

Output: ctx.actions_taken[]

Stage 6 -- Audit (pipeline/stages/audit.py) ? LLM

Role: Write the complete run record to /api/agent-log.

ALWAYS runs -- even on PipelineHalt or unhandled exception.

LLM generates a human-readable narrative for the log.

Functions

?????????

run(ctx, client, llm) -> PipelineContext [called directly from run.py even on halt]

```
_write_run_summary(ctx, client, llm)
    LLM call:
        SYSTEM: "Write a 2-3 sentence plain-English summary of this
                  procurement agent run. Be factual and concise."
        USER:  "Conditions found: {list}
                  Decisions made: {list}
                  Actions taken: {list}
                  Final status: {ctx.final_status}"
    -> narrative (str)
```

```
POST /api/agent-log:
    {trigger: "pipeline_run:{run_id}",
```

```

analysis:    "{n} conditions: {types}",
decision:    "{n} decisions: {actions}",
confidence:  avg(decision.confidence),
metadata:    {run_id, scenario, started_at, duration_ms,
              conditions[], decisions[], actions[],
              final_status, halt_reason,
              llm_narrative,
              financial_safeguards: {max_auto_approve, total_spend}}
_write_pending_decisions(ctx, client)
For each requires_human=True decision:
    POST /api/agent-log (separate entry for dashboard filtering)

API calls:  POST /api/agent-log
LLM:        v plain-English narrative

Output:     ctx.audit_written=True, ctx.final_status="completed|halted|error"

```

```
Any stage raises PipelineHalt("reason", ctx)
    ?
    ? (caught by run.py)
ctx.final_status = "halted"
ctx.halt_reason = reason
Stage 6 (Audit) runs with partial context
run.py returns ctx to caller
```

```

Trigger
?
?
????????????????????????????????????????????????????????????????????????????????????
? Stage 1: INGEST                                                    ?
? GET /inventory · GET /purchase-orders · GET /invoices · GET /suppliers ?
? -> ctx.inventory · ctx.all_pos · ctx.open_pos · ctx.invoices · ctx.suppliers ?
????????????????????????????????????????????????????????????????????????????????????
?
?
????????????????????????????????????????????????????????????????????????????????????
? Stage 2: DETECT (rule-based, no LLM)                                ?
? _detect_duplicate_invoices -> DetectedCondition(duplicate_invoice) ?
? _detect_price_spikes      -> DetectedCondition(price_spike)        ?
? _detect_stockouts         -> DetectedCondition(stockout, critical|warn) ?
? -> ctx.detected_conditions[] <- ALL found; no short-circuit here    ?
????????????????????????????????????????????????????????????????????????????????????
?
?
????????????????????????????????????????????????????????????????????????????????????
? Stage 3: EVALUATE ??? LLM                                           ?
? For each stockout:  score suppliers (rule) + LLM explains choice    ?
? For each price_spike: rank by price; confidence=0.0 (always human)  ?
? For each duplicate:  LLM assesses fraud likelihood                  ?

```

```

? -> ctx.evaluated_items[] ?
????????????????????????????????????????????????????????????????????????????????
?
?
????????????????????????????????????????????????????????????????????????????????
? Stage 4: DECIDE ??? LLM ?
? LLM -> action + confidence + reasoning per item ?
? Code enforces: duplicate/price_spike -> requires_human ?
? PO > $5k -> requires_human ?
? daily spend > $15k -> requires_human ?
? -> ctx.decisions[] ?
????????????????????????????????????????????????????????????????????????????????
?
? APPROVAL GATE ? ??? human: y/n
? (between 4->5) ?
? approved ??? decision.auto_approved = True
? rejected ??? decision.auto_approved = False
????????????????????????????????????????????????????????????????????????????????
?
?
????????????????????????????????????????????????????????????????????????????????
? Stage 5: ACT ?
? auto_approved=True -> POST /purchase-orders, PATCH /{id}/status ?
? requires_human=True -> ActionResult(endpoint="pending_human_review") ?
? -> ctx.actions_taken[] ?
????????????????????????????????????????????????????????????????????????????????
?
?
????????????????????????????????????????????????????????????????????????????????
? Stage 6: AUDIT ??? LLM [ALWAYS RUNS] ?
? LLM -> plain-English narrative ?
? POST /api/agent-log (run summary + per-decision pending entries) ?
? -> ctx.audit_written=True · ctx.final_status ?
????????????????????????????????????????????????????????????????????????????????

```

LLM Integration Summary

Stage	LLM call	Prompt goal	Fallback
3 -- Evaluate	Supplier selection (stockout)	Pick best supplier + explain tradeoff	Top rule
3 -- Evaluate	Duplicate risk (invoice)	Assess fraud vs billing error	confidence = 1.0 if H
4 -- Decide	Action + confidence + reasoning	Recommend action per condition	Hardcoded priori
6 -- Audit	Run narrative	2-3 sentence plain-English summary	Structured JSON summary

All LLM calls use temperature=0.1 (near-deterministic). JSON responses are try/except parsed.

Portability


```
# Install Ollama: https://ollama.com/download
ollama pull mistral

# Override defaults via env vars (no code changes needed)
export OLLAMA_BASE_URL=http://localhost:11434 # or any remote Ollama host
export OLLAMA_MODEL=mistral # or llama3.1, phi3, etc.

# Run
python -m pipeline.run
python -m pipeline.run --approval-mode auto_approve # for testing
```

Uses Ollama's OpenAI-compatible /v1/chat/completions endpoint. Swapping to a different OpenAI-compatible server = change OLLAMA_BASE_URL.

Test Scenario Coverage

Scenario	Detected	Evaluated	Decided	Expected
stockout_warning	stockout(critical, SKU-003)	SUP-003 score=0.920 best	generate_po, requires	
price_spike	price_spike + stockout(SKU-003)	price_spike: confidence=0.0; stockout: SUP-003		
duplicate_invoice	3 duplicate pairs	fraud likelihood assessed	flag_duplicate × 3, requires	
supplier_oos	stockout(SKU-003); SUP-004 excluded	SUP-003 next best (SUP-004 unavailable)		